

STRATEGIC PLANING FOR DATA DRIVEN LOGISTICS OPTIMIZATION

**PROJECT 2 – DATA COLLECTION, CLEANING AND
PREPROCESSING**

PREPARED BY – KHUSHHAL KUMAR SINGH

DATASET USED – DELIVERY LOGISTICS DATA

RECORDS ANALYSED – 25,000

PRIMARY BUSINESS PROBLEM – Reduce anamolies in data, to
reduce any kind of misinterpretation in future business decisions.

TOOLS USED – Python, Pandas, Pycharm, Matplotlib

2. Executive Summary

The quality of data is an important aspect of logistics analytics, as operational choices rely on reliable, complete, consistent, and usable delivery data. The main focus of the given work is to collect, assess, clean, validate, and preprocess the Delivery_Logistics dataset, which contains 25000 delivery records with 15 variables.

The dataset was analysed for missing values, duplicates, duplicate identifiers, inconsistent categorical values, invalid numerical values, constant values, and outliers. The analysis revealed no missing values and no completely identical records. Although 498 records showed the presence of duplicate delivery IDs, there were just an issue with identifier integrity, thus they were kept, as all records contained information about different deliveries rather than duplicates.

The variables of concern, `delivery_time_hours` and `expected_time_hours`, remained constant at the value 00:00.0 for all 25,000 records. As a result these variables were considered not useful and removed from the working dataset. Validation numerically showed that negative values for the distances and the weights of the packages, invalid ratings and negative values of delivery costs were not present in the dataset. The analysis based on IQR method confirmed there were no abnormal observations in the numerical variables.

Lastly, Min-Max normalization was carried on the variables of `distance_km`, `package_weight_kg` and `delivery_cost`, thus providing the values of these variables in the range from 0 to 1 for further statistical modeling and machine learning applications.

Thus, the dataset has been cleaned and prepared for further logistics analysis as well as keeping the original data for reference.

Introduction

Contemporary logistics operations are responsible for creating considerable data because of delivery transactions, transportation processes, customer interaction, and operation control. However, merely obtaining a set of data does not guarantee the fact that it is ready for utilization in analytics. Poor quality of data can result in incorrect performance evaluation, wrong conclusions, and insufficient operational choices.

This project investigates the quality of data as well as the application of preprocessing methods, based on the Delivery_Logistics dataset that was studied in the previous project. The dataset consists of information about delivery employees, types of parcels, types of vehicles used, types of delivery mode, types of geographical locations, weather conditions, delivery distance, weight of the parcel, information about the delivery itself, customer review and delivery expenses.

The main aim of the application of preprocessing methods is not simply to discard unneeded data. Each variable that is available in the dataset must be assessed according to its value for business, analytical period of time, and accuracy level.

Thus, the structured procedure is followed in this project which comprises of:

data collection → data quality assessment → data cleaning → validation → outlier detection → normalization → final dataset creation using Python programming language and libraries such as Pandas and Scikit-learn.

Logistic relevance

Effective delivery data can help in decision-making processes such as:

- detection of delays in delivery,
- choosing the best delivery service providers,
- evaluating logistics costs,
- analysis of the effect of distance and weight of the package,
- evaluation of the efficiency of delivery,
- checking the effectiveness of delivery.

Thus, it can be inferred that improvement of the quality of the data is a good beginning.

3. PROJECT OBJECTIVE

The project has the following primary objectives:

- To gain knowledge regarding the structure and attributes of the Delivery_Logistics dataset.
- To evaluate the dataset for the existence of missing values and duplicates.
- To recognize integrity problems associated with the identifiers.
- To investigate the variables in the dataset for inconsistency.
- To verify if the numerical variables are consistent with reasonable business restrictions.
- To identify outliers statistically by using IQR.
- To eliminate or handle those variables that do not provide any useful information in the analysis.
- To perform numerical normalization using Python.
- To analyze the dataset before and after preprocessing.
- To create the basis of the data for the future logistic projects.

Python code illustration

```
import pandas as pd

# Load the dataset
df = pd.read_csv("Delivery_Logistics(1).csv")

# Display basic information
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nFirst five records:")
print(df.head())
```

Actual Dataset Profile

Attribute	Result
Records	25,000
Original variables	15
Complete duplicate rows	0
Missing values	0
Unique delivery IDs	24,502
Duplicate-ID excess records	498

4. DATA COLLECTION. STIMULATION AND DATASET CHARACTERISTICS

4.1 DATA COLLECTION AND SIMULATION

The data set Delivery_Logistics(1).csv has been tapped for this project to emulate the collection of operational logistics information from a delivery management system. The data represents the various transactional deliveries and gives necessary information regarding transportation, package features, transit performance, customer feedback, and total operational cost.

In real logistics companies, the organization collects such types of data automatically from the use of various logistics applications, GPS-enabled systems, warehouse management systems, and database systems of delivery partners but, for educational purposes, this publicly available data is regarded as the operational raw data before the treatment.

The simulated scenario of data collection describes the receipt of every single delivery transaction done at the moment of order completion and containing information related to the service provider, package type, mode of transport, delivery area, weather, status of delivery, evaluation, and costs associated with delivery. These records form the foundation for analysing the quality of data before conducting business analytics.

4.2 DATASET CHARACTERISTICS

The original dataset contains **25,000 delivery records and 15 variables**.

Table 1: Dataset Overview

Characteristic	Value
Number of records	25,000
Number of variables	15
Complete duplicate rows	0
Missing values	0
Unique delivery IDs	24,502
Repeated-ID records	498

The dataset contains both **categorical and numerical variables**, allowing different data-quality and preprocessing techniques to be demonstrated.

4.3 DESCRIPTION OF VARIABLES

Table 2: Variable Description

Variable	Type	Description
delivery_id	Identifier	Unique reference for a delivery
delivery_partner	Categorical	Company handling the delivery
package_type	Categorical	Category of the delivered package
vehicle_type	Categorical	Vehicle used for transportation
delivery_mode	Categorical	Delivery service type
region	Categorical	Geographical delivery region
weather_condition	Categorical	Weather during delivery
distance_km	Numerical	Delivery distance in kilometres
package_weight_kg	Numerical	Weight of the package
delivery_time_hours	Categorical/Object	Recorded delivery time
expected_time_hours	Categorical/Object	Expected delivery time

delayed	Categorical	Whether the delivery was delayed
delivery_status	Categorical	Final delivery status
delivery_rating	Numerical	Customer delivery rating from 1–5
delivery_cost	Numerical	Cost associated with the delivery

4.4 INITIAL DATA INSPECTION USING PYTHON

The first step was to inspect the dataset structure before making any changes.

Python Code Illustration :

```
import pandas as pd
```

```
# Load the dataset
```

```
df = pd.read_csv("Delivery_Logistics(1).csv")
```

```
# Check dimensions
```

```
print("Dataset shape:", df.shape)
```

```
# Check data types and non-null values
```

```
df.info()
```

```
# Display first five records
```

```
print(df.head())
```


Result

The Python inspection confirmed:

- **25,000 rows**
- **15 columns**
- A combination of numerical, categorical, and identifier variables
- No missing values in the initial dataset

The inspection also highlighted two areas requiring further investigation:

1. `delivery_id` was stored as `float64`, although it functions as an identifier.
2. `delivery_time_hours` and `expected_time_hours` were stored as object variables and required further examination.

These observations were investigated during the subsequent data-quality assessment rather than making assumptions about the data.

4.5 Categorical Data Characteristics

The dataset contains the following numbers of categories:

Variable	Number of Categories
<code>delivery_partner</code>	9
<code>package_type</code>	9
<code>vehicle_type</code>	6
<code>delivery_mode</code>	4
<code>region</code>	5
<code>weather_condition</code>	6
<code>delayed</code>	2
<code>delivery_status</code>	3

The categorical variables were examined for obvious spelling, capitalization, and category inconsistencies. No obvious inconsistent categories were identified, so no categorical values were modified.

4.6 INITIAL DATA COLLECTION FINDINGS

- The primary analysis gives multiple significant insights before starting the cleaning process.
- The data set has a sufficiently big sample of 25,000 delivery transactions which is enough for the logistics analysis.
- There are no missing values in the data which leads to a lesser dependence on imputation.
- The data set does not include fully replicated records.
- The unique identifier issue exists because there are a total of 498 records that share the repeated values of delivery ID.
- Repeated IDs pose a challenge because they have to be studied further and not just erased.
- There are two time-based items in the data set which have the same value for all records, and they have to be analyzed further so that they become valuable for the analysis.
- Numerical variables such as distance, weight, rating and costs also have to be checked for impossibility of values and outliers.
- Hence, the above-mentioned statements show how important data profiling is before starting the cleansing process as the value which poses a technical problem might be a different thing in the business sense.

4.7 Data Preprocessing Roadmap

The following roadmap was adopted to systematically prepare the dataset:

Step 1 — Profile the Raw Dataset



Step 2 — Check Missing Values and Duplicates



Step 3 — Validate Data Types and Identifiers

- ↓
Step 4 — Check Categorical Consistency
- ↓
Step 5 — Validate Numerical Variables
- ↓
Step 6 — Detect Statistical Outliers
- ↓
Step 7 — Remove Non-Informative Variables
- ↓
Step 8 — Normalize Selected Numerical Variables
- ↓
Step 9 — Perform Final Quality Validation
- ↓
Step 10 — Save the Clean Dataset for Further Projects

This roadmap ensures that each preprocessing decision is supported by evidence from the dataset rather than being performed arbitrarily

CONCLUSION

The first analysis indicates that while the Delivery_Logistics dataset is good for the study, there are specific concerns with data quality that need to be taken care of. The main points that need special attention are repeated delivery identifiers and unchanged data related to time. The next phase of the analysis is taking a good look at missing values, duplicates, integrity of identifiers, and data consistency.

5. DATA QUALITY ASSESSMENT

Prior to making any changes in the dataset, data quality assessment was conducted. The aim was to find out about missing values, duplicate records, issues with identifiers, inconsistency in categories and wrong values in numbers and other things that would affect reliability of logistics analysis.

5.1 Missing Value Analysis

Missing values can create incomplete observations and may affect calculations, statistical analysis, and predictive models. Therefore, every variable was checked for missing values.

Python Code Illustration

```
# Check missing values in each column  
missing_values = df.isnull().sum()  
print(missing_values)
```

Result

The analysis showed that **all 15 variables contained zero missing values**.

Data Quality Measure	Result
Total records	25,000
Columns with missing values	0
Total missing values	0

Interpretation

Since there are no missing values, **no imputation or row deletion was required**. This is beneficial because all 25,000 delivery records can be retained for subsequent analysis.

5.2 Complete Duplicate Record Analysis

Duplicate records can result in double-counting and may distort logistics performance measures. Therefore, complete duplicate rows were checked.

Python Code Illustration

```
# Count complete duplicate rows  
duplicate_rows = df.duplicated().sum()  
print("Complete duplicate rows:", duplicate_rows)
```

Result

The dataset contained:

0 complete duplicate rows

Therefore, no records were removed as complete duplicates.

Interpretation

The absence of complete duplicates indicates that every row differs in at least one field. This does **not**, however, mean that every `delivery_id` is unique. Identifier-level duplication was therefore investigated separately.

5.3 Duplicate Delivery ID Analysis

The `delivery_id` variable is intended to identify individual delivery transactions. An identifier should normally be unique. The analysis found **24,502 unique delivery IDs among 25,000 records**, indicating repeated identifiers.

Python Code Illustration

```
# Count repeated delivery IDs
```

```
duplicate_ids = df["delivery_id"].duplicated().sum()
print("Repeated delivery ID records:", duplicate_ids)
print("Unique delivery IDs:", df["delivery_id"].nunique())
```

Result

- Total records = **25,000**
- Unique delivery IDs = **24,502**
- Repeated-ID excess records = **498**

Only two ID values account for this issue:

Delivery ID	Number of Records
250.99	250
24750.01	250

Important Interpretation

These records were **not deleted**.

Although the IDs are repeated, the associated delivery attributes are not identical. For example, records associated with these IDs contain different distances and delivery costs. Therefore, treating them as duplicate transactions and deleting them could incorrectly remove legitimate observations.

The issue is therefore classified as an **identifier-integrity problem rather than a complete-duplicate problem**.

The appropriate treatment is to document the issue and treat `delivery_id` as an identifier rather than as a numerical analytical variable.

5.4 Data Type Assessment

The data types of all variables were reviewed to determine whether they appropriately represented their business meaning.

One important issue was identified with `delivery_id`.

Although it was stored as `float64`, an ID should not be treated as a continuous numerical measurement because mathematical operations on an ID have no business meaning.

Python Code Illustration

```
# Convert delivery ID into a string identifier
```

```
df["delivery_id"] = df["delivery_id"].astype(str)
```

```
print(df["delivery_id"].dtype)
```

The conversion changes the representation of the identifier without deleting any records.

Interpretation

For example, calculating the average of delivery IDs would provide no useful logistics insight. Treating the ID as a string prevents it from accidentally being included in numerical calculations or machine-learning features.

5.5 Constant Variable Assessment

The variables `delivery_time_hours` and `expected_time_hours` were examined because their data type and analytical usefulness required validation.

Both variables contained the same value:

00:00.0

for all **25,000 records**.

Therefore, each variable had only **one unique value**.

Python Code Illustration

```
print(df["delivery_time_hours"].nunique())  
print(df["expected_time_hours"].nunique())  
print(df["delivery_time_hours"].unique())  
print(df["expected_time_hours"].unique())
```

Result

Variable	Unique Values	Observation
delivery_time_hours	1	00:00.0 for all records
expected_time_hours	1	00:00.0 for all records

Interpretation

Because both variables have zero variation, they cannot explain differences between deliveries. Converting them to numerical zero would not restore useful information.

Therefore, they were removed from the **working analytical dataset**, while the original raw dataset was preserved for traceability.

The cleaned base dataset consequently contains:

25,000 records × 13 variables

5.6 Overall Data Quality Findings

The data-quality assessment produced the following results:

Quality Check	Finding	Treatment
Missing values	0	No treatment
Complete duplicate rows	0	No treatment
Unique delivery IDs	24,502	Investigated
Repeated-ID excess records	498	Retained and documented

Categorical inconsistencies	None identified	No treatment
Constant time variables	2	Removed from working dataset
Invalid numerical values	0	No treatment

Key Finding

The dataset should **not simply be labelled as completely clean**. It has excellent completeness at the record level, with no missing values or complete duplicates, but it has an **identifier-integrity issue** and two non-informative constant variables.

This distinction is important because good data cleaning requires understanding the **business meaning of each problem before deciding whether to modify or remove data**.

CONCLUSION

The assessment of data quality indicated that the data set is mainly complete and can be used in the structure. There are no missing values, duplicate complete rows, categorical contradictions, or any problems related to basic validity of numbers identified. The major problems found were the repetition of delivery IDs and repetition of two time-related variables.

Thus, the second stage of the project would be to clean and process the data, with a focus on the problems identified while arriving at the historical information.

6. Data Cleaning and Preprocessing

After completing the data-quality assessment, the identified issues were addressed using a controlled preprocessing approach. The objective was to improve the analytical usability of the dataset **without unnecessarily deleting valid delivery records**.

6.1 Creation of a Working Dataset

The original dataset was preserved so that the raw data remains available for reference and traceability. A separate copy was created for cleaning.

Python Code Illustration

```
# Create a working copy
df_clean = df.copy()
print("Original shape:", df.shape)
print("Working dataset shape:", df_clean.shape)
```

The working dataset initially contained **25,000 records and 15 variables**.

6.2 Correcting the Delivery ID Data Type

The `delivery_id` variable was originally stored as a floating-point number. Since it is an identifier rather than a measurable quantity, it was converted to a string.

Python Code

```
df_clean["delivery_id"] = df_clean["delivery_id"].astype(str)
print(df_clean["delivery_id"].dtype)
```

Reason for Treatment

An identifier should not be used in calculations such as averages or correlations. Converting it to a string makes its role as an identification field explicit.

Important: This conversion does **not** remove the repeated IDs. The 498 repeated-ID records were retained because they were not complete duplicate transactions.

6.3 Removing Non-Informative Variables

The analysis showed that `delivery_time_hours` and `expected_time_hours` contained exactly the same value (00:00.0) for all 25,000 observations.

Since a variable with only one value provides no variation for statistical analysis, both variables were removed from the working dataset.

Python Code

```
df_clean = df_clean.drop(
    columns=["delivery_time_hours", "expected_time_hours"]
)
print("Cleaned dataset shape:", df_clean.shape)
```

Result

The cleaned base dataset contains:

25,000 records × 13 variables

No delivery records were deleted during this process.

6.4 Handling Missing Values

No missing values were identified during the quality assessment. Therefore, no imputation technique was required.

```
print(df_clean.isnull().sum().sum())
```

Result: 0

This means that all remaining observations have values across the 13 retained variables.

6.5 Handling Complete Duplicates

The original dataset contained **zero complete duplicate rows**. Therefore, no duplicate transactions were removed.

```
print("Duplicate rows:", df_clean.duplicated().sum())
```

Result: 0

This confirms that the preprocessing process did not unnecessarily reduce the dataset size.

6.6 HANDLING DUPLICATE IDS

The fact that the IDs kept coming up is not of concern any longer since research has shown that the same IDs have distinct delivery information attributed to them.

This is one of the significant data purging decisions. The deletion of records with duplicate IDs would lead to the loss of usable operational data. The problem can safely be flagged as one requiring source-system verification in a normal logistics situation.

Business Recommendations

The logistics company should verify whether:

- The IDs are valid;
- The IDs are used twice;
- The same ID was assigned to several records;
- The process of preparing the data set made IDs repeat.

Particularly, for the purpose of the project, we are keeping the record in the data set to ensure that none of the possibly good delivery records are lost.

6.7 Final Cleaning Code

The main cleaning operations can be combined into the following Python code:

```
# Create a copy of the raw dataset
```

```
df_clean = df.copy()
```

```
# Treat delivery_id as an identifier
```

```
df_clean["delivery_id"] = df_clean["delivery_id"].astype(str)
```

```
# Remove non-informative constant variables
```

```
df_clean = df_clean.drop(  
    columns=["delivery_time_hours", "expected_time_hours"]  
)
```

```
# Verify final structure
```

```
print("Final shape:", df_clean.shape)
```

```
print("Missing values:", df_clean.isnull().sum().sum())
```

```
print("Duplicate rows:", df_clean.duplicated().sum())
```

Expected Result

Measure	Result
Records	25,000
Variables	13
Missing values	0

Complete duplicate rows	0
Delivery records deleted	0

6.8 Data Cleaning Summary

The preprocessing decisions can be summarized as follows:

Issue	Action	Reason
Missing values	No action	None present
Complete duplicates	No action	None present
Duplicate IDs	Retained and documented	Records contain different information
delivery_id stored as float	Converted to string	Identifier, not numerical measurement
delivery_time_hours	Removed	Constant value
expected_time_hours	Removed	Constant value
Categorical inconsistencies	No action	None identified

KEY FINDING

The primary principle guiding the cleaning process was “clean only when the evidence supports cleaning.” Instead of disregarding records due to multiple instances of an identifier or replacing constant-time values with dummy numeric data, decisions regarding preprocessing were made based on their true analytical and business relevance.

The cleaned dataset is now prepared for the numerical validation and the outlier study, which is the next step in the process.

7. Numerical Data Validation and Outlier Detection

After cleaning the dataset, the numerical variables were examined to ensure that their values were logically valid and that extreme observations were identified before further analysis.

The numerical variables considered were:

- distance_km
- package_weight_kg
- delivery_rating
- delivery_cost

7.1 Numerical Range Validation

Each numerical variable was checked for values that could be considered logically invalid.

Python Code Illustration

```
# Select numerical variables
```

```
numeric_columns = [
```

```
    "distance_km",
```

```
    "package_weight_kg",
```

```
    "delivery_rating",
```

```
    "delivery_cost"
```

```
]
```

```
# Generate descriptive statistics
```

```
print(df_clean[numeric_columns].describe())
```

Actual Results

Variable	Minimum	Maximum	Mean
distance_km	3.6	297.1	150.39
package_weight_kg	0.67	49.52	25.15
delivery_rating	1	5	3.67
delivery_cost	95.67	1,632.72	864.94

These ranges do not indicate obvious invalid values.

7.2 Business Rule Validation

Logical business rules were also applied to the numerical variables.

For a delivery dataset:

- Distance should not be negative.
- Package weight should not be negative.
- Delivery cost should not be negative.
- Delivery rating should remain between 1 and 5.

Python Code

```
print("Negative distances:",  
      (df_clean["distance_km"] < 0).sum())  
  
print("Negative package weights:",  
      (df_clean["package_weight_kg"] < 0).sum())  
  
print("Invalid ratings:",  
      ((df_clean["delivery_rating"] < 1) |
```



```
(df_clean["delivery_rating"] > 5)).sum())
```

```
print("Negative delivery costs:",  
      (df_clean["delivery_cost"] < 0).sum())
```

Actual Results

Validation Rule	Invalid Records
Negative distance	0
Negative package weight	0
Rating outside 1–5	0
Negative delivery cost	0

Interpretation

All four numerical variables passed the basic business-rule validation. Therefore, **no numerical observations were modified or removed** at this stage.

This is important because unnecessary modification of valid operational data could reduce the reliability of subsequent analysis.

7.3 Statistical Outlier Detection

After checking logical validity, statistical outliers were examined using the **Interquartile Range (IQR) method**.

The IQR method identifies observations that fall substantially outside the central 50% of the data.

The formulas used are:

$IQR = Q3 - Q1$
 $Lower\ Bound = Q1 - 1.5(IQR)$
 $Upper\ Bound = Q3 + 1.5(IQR)$

Any observation below the lower bound or above the upper bound is classified as a potential outlier.

7.4 Python Implementation of IQR

for column in numeric_columns:

```
Q1 = df_clean[column].quantile(0.25)
```

```
Q3 = df_clean[column].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
outliers = df_clean[  
    (df_clean[column] < lower_bound) |  
    (df_clean[column] > upper_bound)  
]
```

```
print(column)
```

```
print("Lower bound:", lower_bound)
```

```
print("Upper bound:", upper_bound)
```

```
print("Outliers:", len(outliers))
```

```
print()
```

Actual IQR Results

Variable	Q1	Q3	Lower Bound	Upper Bound	Outliers
distance_km	75.90	224.90	-147.60	448.40	0
package_weight_kg	12.68	37.66	-24.79	75.13	0
delivery_rating	3.00	5.00	0.00	8.00	0
delivery_cost	490.80	1,237.91	-629.87	2,358.58	0

7.5 OUTLIER TREATMENT DECISION

The inspection found no IQR-based anomalies among all four measured variables.

Hence:

- No outlier management was necessary.
- There were no instances of winsorized data or truncation or deletion.
- This is better than falsifying the original sample just to demonstrate some outlier management techniques.

Logistics description

For example, a delivery distance limit of 297.1 km might look a bit high initially. However, it is still less than the determined IQR upper limit of 448.4 km. Thus, it would not qualify as an outlier based on this criterion.\

In the same manner, a delivery expense of ₹1,632.72 lies below the determined IQR maximum of ₹2,358.58.

This is an illustration of the importance of combining business interpretation and statistical analysis when analyzing logistic data.

CONCLUSION

The numerical validation confirms that:

- There are no negative distances observed,
- There are no negative package weights,
- The ratings are valid and fall within the expected range of 1-5,
- All delivery costs are positive,
- There was no detection of outliers based on IQR,
- Thus, no numeric records required any removal or processing

Now the cleaned data can move to the next step, the data normalization process, where different numerical variables will be transformed into a single scale for analysis.

8. DATA NORMALIZATION

Once the numerical variables were cleaned and validated, normalization was applied to bring the chosen variables into conformity. This is particularly important when certain variables are very different in range and will later be analyzed statistically or used in machine learning models.

8.1 Selection of Variables That Were Normalized

Following three variables were chosen for normalization.

- distance_km
- package_weight_kg
- delivery_cost

These variables are from different scales. For instance, distance is measured in kilometers, weight in kilograms, and cost is measured in currency units.

The following values weren't normalized:

- delivery_id - is just a code and not a meaningful measure
- Categorical variables - normalization is not relevant.
- delivery_rating - it is already measured on a fixed scale of 1-5.

8.2 Min-Max Normalization

The **Min-Max normalization** technique was selected as the primary method.

The formula is:

$$X_{\text{normalized}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

This converts each selected variable to a range between **0 and 1**.

Python Code Illustration

```
from sklearn.preprocessing import MinMaxScaler
```

```
normalization_columns = [  
    "distance_km",  
    "package_weight_kg",  
    "delivery_cost"  
]
```

```
scaler = MinMaxScaler()
```

```
for column in normalization_columns:
```

```
    df_clean[column + "_normalized"] = scaler.fit_transform(
```

```
df_clean[[column]]
)
```

The original variables were retained and new normalized variables were created. This preserves the original business values while providing scaled variables for analytical applications.

8.3 Verification of Normalized Values

After normalization, the minimum and maximum values were checked to confirm that the transformation was successful.

```
normalized_columns = [
    "distance_km_normalized",
    "package_weight_kg_normalized",
    "delivery_cost_normalized"
]

print(df_clean[normalized_columns].agg(["min", "max"]))
```

Actual Result

Normalized Variable	Minimum	Maximum
distance_km_normalized	0.00	1.00
package_weight_kg_normalized	0.00	1.00
delivery_cost_normalized	0.00	1.00

This confirms that Min-Max normalization was correctly applied.

8.4 WHY NORMALIZATION IS USEFUL IN LOGISTICS ANALYSIS

Normalization is essential when dealing with multiple numerical variables that are part of the same analytical model.

Suppose:

- Distance from point A to B ranges from 3.6 to 297.1 kilometers.
- Weight of one package is between 0.67 and 49.52 kilograms.
- Cost of the delivery ranges from ₹95.67 to ₹1632.72.
- When dealing with variables that scale differently, models that depend on numerical measurement could give undue weight to variables with larger numerical values.
- With normalization, all three variables are now taken on a common scale of 0 to 1, and can therefore be used for techniques that require feature scaling.

Practical example

Let's say two deliveries have different distances, types of package, and costs. Instead of looking at the raw values, normalized values can be used to evaluate the relative position of these deliveries in the dataset.

In this case, it could be used in future solutions for:

- estimating delivery cost
- segmenting customers or deliveries
- clustering
- predictive modeling
- delivery risk scoring
- machine learning algorithms based on distances

8.5 Alternative: Standardization

As an alternative to Min-Max normalization, **StandardScaler** can be used to standardize variables around a mean of approximately 0 with a standard deviation of approximately 1.

```
from sklearn.preprocessing import StandardScaler
```

```
standard_scaler = StandardScaler()
```

```
standardized_data = standard_scaler.fit_transform(  
    df_clean[normalization_columns]  
)
```

However, **Min-Max normalization was selected as the primary method** for this project because the resulting 0–1 range is straightforward to interpret and preserves the relative ordering of observations

8.6 Final Normalized Dataset

After adding the three normalized variables, the analytical DataFrame contains:

25,000 records × 16 variables

This consists of:

- **13 cleaned base variables**
- **3 normalized variables**

The original raw dataset remains unchanged, while the cleaned and normalized dataset is used for subsequent analysis.

Key Finding

Normalization did not change the underlying delivery information. Instead, it created additional scaled representations of distance, package weight, and delivery cost while retaining their original values for business interpretation.

CONCLUSION

In conclusion, section 8 of the research shows that the process of Min-Max normalization has effectively transformed the three chosen numerical variables into a range of 0-1. This method has improved the suitability of the variables for upcoming statistical and machine-learning processes while keeping the original values untouched.

9. Before-and-After Data Quality Assessment

After completing the cleaning, validation, and normalization processes, the dataset was compared with its original state. This comparison helps demonstrate the actual impact of preprocessing and provides evidence that the cleaning decisions were justified.

9.1 Data Quality Comparison

Data Quality Measure	Before Preprocessing	After Preprocessing	Action Taken
Records	25,000	25,000	All records retained
Variables	15	13	2 non-informative variables removed
Missing values	0	0	No treatment required
Complete duplicate rows	0	0	No treatment required
Unique delivery IDs	24,502	24,502	Issue documented, IDs retained
Repeated-ID excess records	498	498	Not deleted
Categorical inconsistencies	None identified	None identified	No treatment required
Invalid numerical values	0	0	No treatment required

IQR outliers	0	0	No treatment required
Constant time variables	2	0	Removed from analytical dataset

Important Note

The reduction from **15 to 13 variables does not mean that delivery records were lost**. The dataset continues to contain all **25,000 delivery transactions**.

The two removed variables were:

- delivery_time_hours
- expected_time_hours

Both contained only 00:00.0 for every observation and therefore provided no analytical variation.

9.2 Normalization Impact

Normalization created three additional analytical variables:

Original Variable	Normalized Variable	Range After Normalization
distance_km	distance_km_normalized	0–1
package_weight_kg	package_weight_kg_normalized	0–1
delivery_cost	delivery_cost_normalized	0–1

Therefore, the final analytical DataFrame containing the normalized features has:

25,000 records × 16 variables

The 16 variables consist of the **13 cleaned variables plus 3 normalized versions**.

9.3 Python Verification

The final dataset structure was verified using Python.

```
print("Final analytical dataset shape:", df_clean.shape)
```

```
print("\nMissing values:")
```

```
print(df_clean.isnull().sum().sum())
```

```
print("\nComplete duplicate rows:")
```

```
print(df_clean.duplicated().sum())
```

```
print("\nNormalized variable ranges:")
```

```
print(df_clean[[
    "distance_km_normalized",
    "package_weight_kg_normalized",
    "delivery_cost_normalized"
]].agg(["min", "max"]))
```

Expected Output

Final analytical dataset shape: (25000, 16)

Missing values: 0

Complete duplicate rows: 0

The normalized variables have minimum values of **0.00** and maximum values of **1.00**.

9.4 General Evaluation

Preprocessing enabled to extract more value from the dataset.

The main results of preprocessing are:

- No records of deliveries were deleted unnecessarily.
- Problems with missing values were eliminated at the beginning.
- There were none of the identical transactions present.
- Repeated delivery IDs were considered a problem of identifier integrity and not as a deletion of supposedly valid transactions.
- Two variables that did not have any information value were eliminated.
- There were no invalid numerical values.
- No outliers were found using the IQR technique.
- Three numerical variables were rescaled in the range of 0–1.

The conclusion

The dataset can be deemed analytically ready but not entirely free from quality issues. Repeated delivery IDs warrant further investigations at the level of the source system in the actual logistics operation.

It is important to note that preprocessing has added analytical value without resolving the apparent data integrity problem.

9.5 Overview of Data Quality Improvement Plan

Based on the results, we propose the following plan for the real-life logistics company

- Immediate – Keep compulsory checks for delivery ID, length, weight, grade, and cost.
- Short-Term – Explore the reason behind repetitive delivery IDs and introduce automated verification of their uniqueness.
- Mid-Term – Do not let irrelevant and always-not-working fields get into analytical datasets unless they provide meaningful information

- Long-Term – Set up continuous automated dashboards monitoring completeness, uniqueness, validity, consistency, and outliers.
- This plan will prevent similar data quality problems from occurring in logistics data in the future.

CONCLUSION

What the before-and-after evaluation shows is that the preprocessing was based on evidence and was conservative. The entire set of 25,000 records was retained while the non-informative variables were deleted.

As a result, the dataset is now ready for logistics analysis, visualization, statistical modeling, and machine learning implementation in the future projects.

10. Practical Logistics Demonstration and Business Impact

The purified data set is now able to be applied in practice demonstrating how data quality impacts making logistics decisions. Data preprocessing is important because flawed or duplicated or irrelevant data can lead to erroneous operational conclusions.

10.1 Example: Distance and Cost of Delivery

The data set shows that:

- Average distance of delivery = 150.39 km
- Maximum distance of delivery = 297.1 km
- Average cost of delivery = 864.94
- Maximum cost of delivery = 1,632.72

Distance and cost of delivery are measured on different scales. Consequently, normalization of values produces similar normalized values from 0 to 1 that can be used later in operations with analytical models.

As an example, logistics companies may input normalized distance and cost into a delivery cost prediction model.

10.2: Package Weight

The average weight of a package is about 25.15 kg, which is close to the maximum measured weight of 49.52 kg.

The weight of packages can impact decisions around the transportation as heavier packages may need:

- bigger vehicle,
- other arrangements,
- additional handling, and
- higher costs of delivery.

Due to the dataset not having negative package weights or outliers based on the IQR, the direct weights were employed instead of modifying the results artificially.

10.3: Customer Delivery Ratings

The delivery_rating variable has values from 1 to 5 with average rating being around 3.67.

In future projects this variable will help in exploring the influence of customer satisfaction on operational attributes like:

- the delivery partner,
- the delivery type,
- the weather conditions,
- the type of vehicle, and
- the status of delivery.

No normalization was applied to the rating, as the fact that it is always between 1 and 5 means that it is already understandable value.

10.4 Example of Delivery IDs

- The problem of repeated delivery IDs raises a serious concern in this context.
- Total number of records with repeated IDs amounts to 498; however, these records cannot be deleted due to existence of other attributes.
- From the business viewpoint, removing them from the database uninvestigated would mean missing several real transactions.
- Logistics companies should thus apply the automated validation rule as follows:
- Each new transaction has to be attributed a unique and valid delivery ID.

Records failing to satisfy this condition could be flagged for further investigation prior to including them into the analytical database.

10.5 Impact of Data Quality on Logistics Analytics

Poor data quality can directly affect logistics decisions.

Data Quality Problem	Potential Logistics Impact
Duplicate transactions	Inflated delivery counts and misleading performance metrics
Incorrect IDs	Difficulty tracking individual deliveries
Missing values	Incomplete analysis and unreliable models
Invalid distance/weight	Incorrect transportation planning
Invalid costs	Incorrect cost estimates
Non-informative variables	Unnecessary complexity in analytical models

Unscaled numerical variables	Potential bias in scale-sensitive algorithms
------------------------------	--

In this dataset, most of these issues were **not present**, which is a positive finding. However, the repeated-ID issue demonstrates why identifier validation remains important even when the dataset has no missing values or complete duplicates.

10.6 Recommendations

From the data quality assessment, the following recommendations can be made:

- Conduct unique ID verification when generating a record of delivery.
- Perform automated data quality verification to detect undesired data.
- Keep identifiers separate from analysis variables during the data studies.
- Watch out for fields with zero variation in order to avoid entering data void of information for analysis.
- Maintain raw data separately so that its cleaning process is transparent and reversible.
- Make normalization or standardization whenever it is required by the chosen data analysis methodology.
- Establish data quality board to ensure permanent monitoring of logistics data.

The above recommendations will allow changing the practice of data cleaning from a one-time action into a process of on-going data governance.

CONCLUSION

The result of the preprocessing indicates that the quality of data has a direct link with the trustworthiness of logistics analytics. The Delivery_Logistics dataset is quite complete and valid from an overall perspective. However, data completeness does not assure data integrity, as shown by the presence of duplicates of delivery IDs.

11. Reflection

This project taught hands-on experience regarding the influence of data quality on the accuracy of logistics analytics. Instead of assuming that the solution to data problems lies in deleting data records, we carefully examined the dataset and established the importance of each problem in terms of its analytical and business value.

One of the key lessons learned from this project was that data completeness and data integrity are two different things. The dataset used in this project had no missing values and no identical rows, yet it still contained 498 records with the same ID. This proved that something looking clean from the point of view of data completeness may still have some problems in terms of data integrity.

Another significant lesson was the importance of knowing the meaning of variables before preprocessing them. The delivery_id variable was presented as number, but after due consideration, it had been correctly interpreted as an identifier and converted into a string. Likewise, two time-related variables were not transformed into artificial numbers since they had only 00:00.0 and no analytical variability

Major learning outcomes

From the completion of this project, I gained knowledge and experience in:

- Data profiling using Python and Pandas.

- Identification of data quality problems and interpretation of the findings.
- Data analysis related to missing values and duplicates.
- Validation of identifiers and recognition of the data type.
- Validation of the numerical business rule.
- IQR outlier detection.
- Min-Max normalization through Scikit-learn.
- Evidence-based decisions with regard to data cleaning.
- Understanding how data quality impacts logistics decision making.

Assessment

An important outcome of the project was the understanding of the necessity to keep the raw data separate from the cleaned data to ensure reproducibility of preprocessing and enable the analyst to track each change.

It should be highlighted that an important conclusion that can be derived from the project is that efficient data preprocessing is not only about making the dataset smaller or dealing with the most extreme values, but rather focusing on decisions that center around improving analysis effectiveness and maintaining legitimate business information.

CONCLUSION

The project examined and prepared the Delivery_Logistics data for future logistical evaluation.

The raw data includes 25,000 case records with 15 defining variables. During the examination, it was determined that there were no missing data, no duplicate rows and no erroneous data. However, 498 rows had identical delivery IDs, indicating a problem with the delivery ID's integrity but allowing for the records to be kept.

Variables of delivery_time_hours and expected_time_hours have been excluded from the cleaned dataset due to the lack of variability between the identified values. None of the variables were found to be erroneous while the examination of the IQR measure revealed no outlying data.

Lastly, the normalized data was obtained for variables of distance_km, package_weight_kg and delivery_cost after the help of Min-Max normalization.

The cleaned data preserves all 25,000 records and has been enhanced with the normalized variables in order to improve the quality of the analysis.

The key conclusion is that data quality should not be viewed as a one-time cleaning process but as a continuous activity. The deployment of automated validation, unique-ID optimization, data-quality monitoring, and documentation should be part of the real logistics systems.

13. REFERENCES

Pandas documentation reference. Pandas: Python Data Analysis Library. It is the library that helps in loading data, reviewing and analyzing the existence of missing values, removing duplicate values, figuring out data types, etc. Scikit-learn documentation reference. Preprocessing. It is the library that uses Min-Max normalization and StandardScaler.

Python software foundation. Python reference. The library that is the programming language for how the process of data preprocessing is carried out. Delivery_Logistics data set. Delivery_Logistics(1).csv. The reference is the major data source used in the data quality, cleaning, validation, outlier knowledge, and normalization activities.

If your university mandates the referencing to follow APA citation style, the technical references may be presented as follows:

- pandas development team. (n.d.). pandas documentation. pandas.
- scikit-learn development team. (n.d.). Preprocessing. scikit-learn.
- Python software foundation. (n.d.). Python documentation.
Python.org
- Delivery_Logistics. (n.d.). Delivery logistics data set, the CSV file.

14. Appendix A — Complete Python Code

This section should be placed at the end of your report.

It contains the complete code used throughout Project 2, while the shorter code snippets can remain within the relevant report sections.

```
#
=====

=====

# PROJECT 2 - DATA COLLECTION, CLEANING AND
PREPROCESSING

# Dataset: Delivery_Logistics

#
=====

=====

import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# -----
# 1. LOAD DATASET
# -----

df = pd.read_csv("Delivery_Logistics(1).csv")

print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
```

```
print("\nFirst five records:")  
print(df.head())
```

```
print("\nData types:")  
print(df.dtypes)
```

```
# -----
```

2. MISSING VALUE ANALYSIS

```
# -----
```

```
print("\nMissing values:")  
print(df.isnull().sum())
```

```
print("\nTotal missing values:",  
      df.isnull().sum().sum())
```

```
# -----
```

3. COMPLETE DUPLICATE ANALYSIS

```
# -----
```

```
print("\nComplete duplicate rows:",  
      df.duplicated().sum())
```

```
# -----
```

4. DELIVERY ID ANALYSIS

```
# -----
```

```
print("\nUnique delivery IDs:",  
      df["delivery_id"].nunique())
```

```
print("Repeated delivery-ID records:",  
      df["delivery_id"].duplicated().sum())
```

```
print("\nRepeated ID values:")  
print(df["delivery_id"].value_counts().head())
```

```
# -----
```

5. CATEGORICAL VARIABLE INSPECTION

```
# -----
```

```
categorical_columns = [  
    "delivery_partner",  
    "package_type",  
    "vehicle_type",  
    "delivery_mode",  
    "region",  
    "weather_condition",  
    "delayed",
```

```
"delivery_status"
]

for column in categorical_columns:
    print("\n", column)
    print(df[column].unique())
    print("Number of categories:",
          df[column].nunique())

# -----
# 6. CHECK CONSTANT VARIABLES
# -----

print("\nUnique values in delivery_time_hours:")
print(df["delivery_time_hours"].nunique())

print("\nUnique values in expected_time_hours:")
print(df["expected_time_hours"].nunique())

print("\nDelivery time values:")
print(df["delivery_time_hours"].unique())

print("\nExpected time values:")
print(df["expected_time_hours"].unique())
```



```
# -----
```

```
# 7. CREATE CLEAN WORKING DATASET
```

```
# -----
```

```
df_clean = df.copy()
```

```
# Treat delivery ID as an identifier
```

```
df_clean["delivery_id"] = (  
    df_clean["delivery_id"].astype(str)  
)
```

```
# Remove non-informative constant variables
```

```
df_clean = df_clean.drop(  
    columns=[  
        "delivery_time_hours",  
        "expected_time_hours"  
    ]  
)
```

```
print("\nCleaned base dataset shape:",  
      df_clean.shape)
```

```
# -----
```

```
# 8. NUMERICAL VARIABLES
```

```
# -----
```

```
numeric_columns = [  
    "distance_km",  
    "package_weight_kg",  
    "delivery_rating",  
    "delivery_cost"  
]
```

```
print("\nDescriptive statistics:")  
print(df_clean[numeric_columns].describe())
```

```
# -----
```

9. BUSINESS-RULE VALIDATION

```
# -----
```

```
print("\nNegative distances:",  
      (df_clean["distance_km"] < 0).sum())
```

```
print("Negative package weights:",  
      (df_clean["package_weight_kg"] < 0).sum())
```

```
print("Invalid ratings:",  
      ((df_clean["delivery_rating"] < 1) |  
       (df_clean["delivery_rating"] > 5)).sum())
```

```
print("Negative delivery costs:",  
      (df_clean["delivery_cost"] < 0).sum())
```

```
# -----
```

```
# 10. IQR OUTLIER DETECTION
```

```
# -----
```

```
for column in numeric_columns:
```

```
    Q1 = df_clean[column].quantile(0.25)
```

```
    Q3 = df_clean[column].quantile(0.75)
```

```
    IQR = Q3 - Q1
```

```
    lower_bound = Q1 - 1.5 * IQR
```

```
    upper_bound = Q3 + 1.5 * IQR
```

```
    outliers = df_clean[  
        (df_clean[column] < lower_bound) |  
        (df_clean[column] > upper_bound)  
    ]
```

```
    print("\nColumn:", column)
```

```
    print("Q1:", Q1)
```

```
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)
print("Number of outliers:", len(outliers))
```

```
# -----
```

```
# 11. MIN-MAX NORMALIZATION
```

```
# -----
```

```
normalization_columns = [
    "distance_km",
    "package_weight_kg",
    "delivery_cost"
]
```

```
scaler = MinMaxScaler()
```

```
for column in normalization_columns:
```

```
    df_clean[column + "_normalized"] = (
        scaler.fit_transform(
            df_clean[[column]]
        )
    )
```

```
print("\nNormalized data:")
print(
    df_clean[
        [
            "distance_km_normalized",
            "package_weight_kg_normalized",
            "delivery_cost_normalized"
        ]
    ].head()
)

# -----
# 12. VERIFY NORMALIZATION
# -----

normalized_columns = [
    "distance_km_normalized",
    "package_weight_kg_normalized",
    "delivery_cost_normalized"
]

print("\nNormalized variable ranges:")
print(
    df_clean[normalized_columns].agg(["min", "max"])
```

```
)
```

```
# -----
```

```
# 13. OPTIONAL STANDARDIZATION
```

```
# -----
```

```
standard_scaler = StandardScaler()
```

```
standardized_data = standard_scaler.fit_transform(  
    df_clean[normalization_columns]  
)
```

```
print("\nStandardized data:")
```

```
print(standardized_data[:5])
```

```
# -----
```

```
# 14. FINAL DATA QUALITY CHECK
```

```
# -----
```

```
print("\n===== FINAL CHECK =====")
```

```
print("Final analytical dataset shape:",  
    df_clean.shape)
```

```
print("Total missing values:",  
      df_clean.isnull().sum().sum())
```

```
print("Complete duplicate rows:",  
      df_clean.duplicated().sum())
```

```
print("\nFinal column list:")  
print(df_clean.columns.tolist())
```

```
# -----  
# 15. SAVE CLEANED DATASET  
# -----
```

```
df_clean.to_csv(  
    "Delivery_Logistics_Cleaned.csv",  
    index=False  
)
```

```
print("\nCleaned dataset saved successfully.")
```

